JOIN, GROUP BY

```
mysql> create database calibojoin;
in;
Query OK, 1 row affected (0.01 sec)

mysql> use calibojoin;
Database changed
mysql>
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| basicsqltable      |
| calibo             |
| calibojoin         |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
7 rows in set (0.00 sec)

mysql>
```

INSERT DATA BY RUNNING .SQL file

BUT THE COLUMN NAME DIFFER IN DATA set given
SO INSERT YOUR OWN DATA AND CLOUMN NAMES

```
## Database Schema Overview
The database contains the following tables:
**departments**
- dept_id (Primary Key)
- dept_name
- location
**employees**
- employee_id (Primary Key)
- first_name
- last_name
- email
- hire_date
- salary
- dept_id (Foreign Key → departments)
- manager_id (Foreign Key → employees)
**customers**
- customer_id (Primary Key)
- customer_name
- email
- city
- country
**orders**
- order_id (Primary Key)
- customer_id (Foreign Key → customers)
- order_date
- total_amount
- status
**products**
- product_id (Primary Key)
- product_name
- category
- unit_price
**order_details**
- order_detail_id (Primary Key)
- order_id (Foreign Key → orders)
- product_id (Foreign Key → products)
- quantity
- unit_price
## Setup Instructions
Before starting the exercises, run the setup script provided by your trainer to create the database tables and insert sample data.
## Exercises
*/
create database if not exists caliboo_db;
use caliboo_db;

DROP TABLE IF EXISTS order_details;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS customers;
DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS departments;

create table departments(
dept_id int primary key,
dept_name varchar(50),
location varchar(50)
);
create table employees(
employee_id int primary key,
first_name varchar(50),
last_name varchar(50),
email varchar(50),
hire_date date,
```

```
salary int,
dept_id int,
manager_id int,
foreign key(dept_id) references departments(dept_id),
foreign key(manager_id) references employees(employee_id)
);
create table customers(
customer_id int primary key,
customer_name varchar(50),
email varchar(50),
city varchar(50),
country varchar(50)
);
create table orders(
order_id int primary key,
customer_id int,
order_date date,
total_amount decimal(10,2),
status varchar(20),
foreign key(customer_id) references customers(customer_id)
);
create table products(
product_id int primary key,
product_name varchar(50),
category varchar(50),
unit_price decimal(10,2)
);
create table order_details(
order_detail_id int primary key,
order_id int,
product_id int,
quantity int,
unit_price decimal(10,2),
foreign key(order_id) references orders(order_id),
foreign key(product_id) references products(product_id)
);
insert into departments values
(1,'HR','Delhi'),
(2,'IT','Bangalore'),
(3,'Sales','Mumbai'),
(4,'Finance','Pune');
insert into employees values
(101,'Amit','Sharma','amit@gmail.com','2021-01-10',80000,2,null),
(102,'Priya','Verma','priya@gmail.com','2021-03-15',65000,2,101),
(103,'Rahul','Mehta','rahul@gmail.com','2022-02-20',72000,2,101),
(104,'Sneha','Reddy','sneha@gmail.com','2022-05-12',48000,1,101),
(105,'Kiran','Patel','kiran@gmail.com','2023-01-05',52000,1,101),
(106,'Neha','Singh','neha@gmail.com','2023-03-08',60000,1,101),
(107,'Arjun','Iyer','arjun@gmail.com','2022-06-10',90000,3,101),
(108,'Rohit','Kumar','rohit@gmail.com','2023-02-15',55000,3,107),
(109,'Pooja','Nair','pooja@gmail.com','2023-04-01',75000,4,101);
insert into customers values
(1,'Amit Sharma','amit@gmail.com','Delhi','India'),
(2,'Priya Verma','priya@gmail.com','Mumbai','India'),
(3,'Rahul Mehta','rahul@gmail.com','Ahmedabad','India'),
(4,'Sneha Reddy','sneha@gmail.com','Hyderabad','India'),
(5,'Kunal Shah','kunal@gmail.com','Surat','India');
insert into orders values
(1001,1,'2024-01-10',2500,'Completed'),
(1002,1,'2024-02-15',1800,'Completed'),
(1003,2,'2024-03-05',3200,'Pending'),
(1004,3,'2024-02-20',1500,'Completed'),
(1005,3,'2024-03-22',2100,'Completed');
insert into products values
(201,'Keyboard','Electronics',500),
(202,'Mouse','Electronics',300),
(203,'Monitor','Electronics',8000),
(204,'Notebook','Stationery',100),
(205,'Pen','Stationery',20);
insert into order_details values
(1,1001,201,2,500),
(2,1001,202,1,300),
(3,1002,201,3,500),
(4,1003,203,1,8000),
(5,1004,204,5,100),
(6,1005,201,1,500),
(7,1005,202,2,300);
```

```
mysql> create database caliboo_db;
Query OK, 1 row affected (0.03 sec)

mysql> use caliboo_db;
Database changed
mysql> source SQL\ Database.sql;
```

```
TERMINAL

Query OK, 0 rows affected (0.06 sec)

Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

Query OK, 9 rows affected (0.01 sec)
Records: 9  Duplicates: 0  Warnings: 0

Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0

Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

Query OK, 7 rows affected (0.01 sec)
Records: 7  Duplicates: 0  Warnings: 0

mysql>
```

```
-- ### Exercise 1: Basic INNER JOIN
-- List all employees with their department names. Display employee ID,
-- first name, last name, and department name. Order the results by employee ID.
-- **Hint:** You need to join the employees and departments tables.
-- **Expected columns:** employee_id, first_name, last_name, dept_name


select e.employee_id,e.first_name,e.last_name,d.dept_name
from employees e
join departments d on e.dept_id=d.dept_id
order by e.employee_id;
```

```
mysql> source exercise1.sql;
+-------------+------------+-----------+-----------+
| employee_id | first_name | last_name | dept_name |
+-------------+------------+-----------+-----------+
|         101 | Amit       | Sharma    | IT        |
|         102 | Priya      | Verma     | IT        |
|         103 | Rahul      | Mehta     | IT        |
|         104 | Sneha      | Reddy     | HR        |
|         105 | Kiran      | Patel     | HR        |
|         106 | Neha       | Singh     | HR        |
|         107 | Arjun      | Iyer      | Sales     |
|         108 | Rohit      | Kumar     | Sales     |
|         109 | Pooja      | Nair      | Finance   |
+-------------+------------+-----------+-----------+
9 rows in set (0.00 sec)

mysql>
```

```
-- ### Exercise 2: LEFT JOIN to Find Missing Data
-- Find all customers and their order count, including customers who haven't placed any orders.
-- Display customer ID, customer name, and the count of orders. Order by order count in descending order.
-- **Hint:** Use LEFT JOIN to include customers with no orders, and COUNT with GROUP BY.
-- **Expected columns:** customer_id, customer_name, order_count


select c.customer_id,c.customer_name,
count(o.order_id) as order_count
from customers c
left join orders o on c.customer_id=o.customer_id
group by c.customer_id,c.customer_name
order by order_count desc;
```

```
mysql> source exercise2.sql;
+-------------+---------------+-------------+
| customer_id | customer_name | order_count |
+-------------+---------------+-------------+
|           1 | Amit Sharma   |           2 |
|           3 | Rahul Mehta   |           2 |
|           2 | Priya Verma   |           1 |
|           4 | Sneha Reddy   |           0 |
|           5 | Kunal Shah    |           0 |
+-------------+---------------+-------------+
5 rows in set (0.01 sec)
```

exercise2.sql

```sql
-- ### Exercise 2: LEFT JOIN to Find Missing Data
-- Find all customers and their order count, including customers who haven't placed any orders.
-- Display customer ID, customer name, and the count of orders. Order by order count in descending order.
-- **Hint:** Use LEFT JOIN to include customers with no orders, and COUNT with GROUP BY.
-- **Expected columns:** customer_id, customer_name, order_count
select c.customer_id,c.customer_name,
count(o.order_id) as order_count
from customers c
left join orders o on c.customer_id=o.customer_id
group by c.customer_id,c.customer_name
order by order_count desc;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
mysql> source exercise2.sql;
+-------------+---------------+-------------+
| customer_id | customer_name | order_count |
+-------------+---------------+-------------+
|           1 | Amit Sharma   |           2 |
|           3 | Rahul Mehta   |           2 |
|           2 | Priya Verma   |           1 |
|           4 | Sneha Reddy   |           0 |
|           5 | Kunal Shah    |           0 |
+-------------+---------------+-------------+
5 rows in set (0.01 sec)

mysql>
```

```sql
-- ### Exercise 3: Multiple JOINs
-- List all orders with customer names and the total number of items in each order. Display order ID
-- , customer name, order date, total amount, count of different items, and sum of quantities. Order by order ID.
-- **Hint:** You need to join three tables: orders, customers, and order_details.
-- **Expected columns:** order_id, customer_name, order_date, total_amount, item_count, total_quantity

select o.order_id,c.customer_name,o.order_date,o.total_amount,
count(distinct od.product_id) as item_count,
sum(od.quantity) as total_quantity
from orders o
join customers c on o.customer_id=c.customer_id
join order_details od on o.order_id=od.order_id
group by o.order_id,c.customer_name,o.order_date,o.total_amount
order by o.order_id;
```

//When you use COUNT, SUM, etc., **all other selected columns must be in GROUP BY**. Should use group by

exercise3.sql

```sql

-- ### Exercise 3: Multiple JOINs
-- List all orders with customer names and the total number of items in each order. Display order ID
-- , customer name, order date, total amount, count of different items, and sum of quantities. Order by order ID.
-- **Hint:** You need to join three tables: orders, customers, and order_details.
-- **Expected columns:** order_id, customer_name, order_date, total_amount, item_count, total_quantity

select o.order_id,c.customer_name,o.order_date,o.total_amount,
count(distinct od.product_id) as item_count,
sum(od.quantity) as total_quantity
from orders o
join customers c on o.customer_id=c.customer_id
join order_details od on o.order_id=od.order_id
group by o.order_id,c.customer_name,o.order_date,o.total_amount
order by o.order_id;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
5 rows in set (0.00 sec)

mysql> source exercise3.sql;
+----------+---------------+------------+--------------+------------+----------------+
| order_id | customer_name | order_date | total_amount | item_count | total_quantity |
+----------+---------------+------------+--------------+------------+----------------+
|     1001 | Amit Sharma   | 2024-01-10 |      2500.00 |          2 |              3 |
|     1002 | Amit Sharma   | 2024-02-15 |      1800.00 |          1 |              3 |
|     1003 | Priya Verma   | 2024-03-05 |      3200.00 |          1 |              1 |
|     1004 | Rahul Mehta   | 2024-02-20 |      1500.00 |          1 |              5 |
|     1005 | Rahul Mehta   | 2024-03-22 |      2100.00 |          2 |              3 |
+----------+---------------+------------+--------------+------------+----------------+
5 rows in set (0.00 sec)

mysql>
```

```sql
-- ### Exercise 4: GROUP BY with Aggregates
-- Calculate the average salary by department. Display department name, count of employees, average salary, minimum salary,
-- and maximum salary. Order by average salary in descending order.
-- **Hint:** Join departments and employees, then use GROUP BY with aggregate functions.
-- **Expected columns:** dept_name, employee_count, avg_salary, min_salary, max_salary
```

```
select 'exercise 4' as '';
SELECT d.dept_name,
COUNT(e.employee_id) AS employee_count,
AVG(e.salary) AS avg_salary,
MIN(e.salary) AS min_salary,
MAX(e.salary) AS max_salary
FROM departments d
JOIN employees e ON d.dept_id=e.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary DESC;
```

exercise4.sql

```
-- ### Exercise 4: GROUP BY with Aggregates
-- Calculate the average salary by department. Display department name, count of employees, average salary, minimum salary,
-- and maximum salary. Order by average salary in descending order.
-- **Hint:** Join departments and employees, then use GROUP BY with aggregate functions.
-- **Expected columns:** dept_name, employee_count, avg_salary, min_salary, max_salary
select 'exercise 4' as "";
SELECT d.dept_name,
COUNT(e.employee_id) AS employee_count,
AVG(e.salary) AS avg_salary,
MIN(e.salary) AS min_salary,
MAX(e.salary) AS max_salary
FROM departments d
JOIN employees e ON d.dept_id=e.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary DESC;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
mysql> source exercise4.sql;
+------------+
|            |
+------------+
| exercise 4 |
+------------+
1 row in set (0.00 sec)

+-----------+---------------+------------+------------+------------+
| dept_name | employee_count | avg_salary | min_salary | max_salary |
+-----------+---------------+------------+------------+------------+
| Finance   |             1 | 75000.0000 |      75000 |      75000 |
| Sales     |             2 | 72500.0000 |      55000 |      90000 |
| IT        |             3 | 72333.3333 |      65000 |      80000 |
| HR        |             3 | 53333.3333 |      48000 |      60000 |
+-----------+---------------+------------+------------+------------+
4 rows in set (0.00 sec)

mysql>
```

```
-- ### Exercise 5: HAVING Clause
-- Find departments with an average salary greater than 65000 and more than 2 employees. Display
-- department name, employee count, and average salary. Order by average salary in descending order.
-- **Hint:** Use HAVING clause to filter groups after aggregation.
-- **Expected columns:** dept_name, employee_count, avg_salary


select 'exercise 5' as '';
select d.dept_name as dept_name,
count(e.employee_id) as employee_count,
avg(e.salary) as avg_salary
from departments d,employees e
where d.dept_id=e.dept_id
group by d.dept_name
having avg(e.salary)>65000
and count(e.employee_id)>2
order by avg_salary desc;
```

exercise5.sql

```
-- ### Exercise 5: HAVING Clause
-- Find departments with an average salary greater than 65000 and more than 2 employees. Display
-- department name, employee count, and average salary. Order by average salary in descending order.
-- **Hint:** Use HAVING clause to filter groups after aggregation.
-- **Expected columns:** dept_name, employee_count, avg_salary
select 'exercise 5' as '';
select d.dept_name as dept_name,
count(e.employee_id) as employee_count,
avg(e.salary) as avg_salary
from departments d,employees e
where d.dept_id=e.dept_id
group by d.dept_name
having avg(e.salary)>65000
and count(e.employee_id)>2
order by avg_salary desc;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
mysql> source exercise5.sql;
+------------+
|            |
+------------+
| exercise 5 |
+------------+
1 row in set (0.00 sec)

+-----------+---------------+------------+
| dept_name | employee_count | avg_salary |
+-----------+---------------+------------+
| IT        |             3 | 72333.3333 |
+-----------+---------------+------------+
1 row in set (0.00 sec)
```

```
-- ### Exercise 6: SELF JOIN
-- List all employees with their manager's name. Display employee ID,
-- employee full name, employee salary, manager full name, and manager salary. Order by employee ID.
-- **Hint:** Join the employees table to itself. Use LEFT JOIN to include employees without managers.
-- **Expected columns:** employee_id, employee_name, employee_salary, manager_name, manager_salary

select 'exercise 6' as '';
select e.employee_id as employee_id,
concat(e.first_name,' ',e.last_name) as employee_name,
e.salary as employee_salary,
concat(m.first_name,' ',m.last_name) as manager_name,
m.salary as manager_salary
from employees e
left join employees m on e.manager_id=m.employee_id
order by e.employee_id;
```

exercise6.sql × SQL Database.sql

exercise6.sql

```
1
2   -- ### Exercise 6: SELF JOIN
3   -- List all employees with their manager's name. Display employee ID,
4   -- employee full name, employee salary, manager full name, and manager salary. Order by employee ID.
5   -- **Hint:** Join the employees table to itself. Use LEFT JOIN to include employees without managers.
6   -- **Expected columns:** employee_id, employee_name, employee_salary, manager_name, manager_salary
7   select 'exercise 6' as '';
8   select e.employee_id as employee_id,
9   concat(e.first_name,' ',e.last_name) as employee_name,
10  e.salary as employee_salary,
11  concat(m.first_name,' ',m.last_name) as manager_name,
12  m.salary as manager_salary
13  from employees e
14  left join employees m on e.manager_id=m.employee_id
15  order by e.employee_id;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
+------------+
|            |
+------------+
| exercise 6 |
+------------+
1 row in set (0.00 sec)

+-------------+--------------+----------------+--------------+----------------+
| employee_id | employee_name | employee_salary | manager_name | manager_salary |
+-------------+--------------+----------------+--------------+----------------+
|         101 | Amit Sharma  |          80000 | NULL         |           NULL |
|         102 | Priya Verma  |          65000 | Amit Sharma  |          80000 |
|         103 | Rahul Mehta  |          72000 | Amit Sharma  |          80000 |
|         104 | Sneha Reddy  |          48000 | Amit Sharma  |          80000 |
|         105 | Kiran Patel  |          52000 | Amit Sharma  |          80000 |
|         106 | Neha Singh   |          60000 | Amit Sharma  |          80000 |
|         107 | Arjun Iyer   |          90000 | Amit Sharma  |          80000 |
|         108 | Rohit Kumar  |          55000 | Arjun Iyer   |          90000 |
|         109 | Pooja Nair   |          75000 | Amit Sharma  |          80000 |
+-------------+--------------+----------------+--------------+----------------+
9 rows in set (0.01 sec)
```

```
-- ### Exercise 7: Complex Aggregation
-- Find the total revenue by product category. Display category, count of distinct orders,
-- total units sold, and total revenue. Order by total revenue in descending order.
-- **Hint:** Join products and order_details, calculate revenue as quantity × unit_price.
-- **Expected columns:** category, order_count, total_units_sold, total_revenue


select 'exercise 7' as '';
select p.category as category,
count(distinct od.order_id) as order_count,
sum(od.quantity) as total_units_sold,
sum(od.quantity*od.unit_price) as total_revenue
from products p
join order_details od on p.product_id=od.product_id
group by p.category
order by total_revenue desc;
```

exercise7.sql × SQL Database.sql

exercise7.sql

```sql
-- ### Exercise 7: Complex Aggregation
-- Find the total revenue by product category. Display category, count of distinct orders,
-- total units sold, and total revenue. Order by total revenue in descending order.
-- **Hint:** Join products and order_details, calculate revenue as quantity × unit_price.
-- **Expected columns:** category, order_count, total_units_sold, total_revenue
select 'exercise 7' as '';

select p.category as category,
count(distinct od.order_id) as order_count,
sum(od.quantity) as total_units_sold,
sum(od.quantity*od.unit_price) as total_revenue
from products p
join order_details od on p.product_id=od.product_id
group by p.category
order by total_revenue desc;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
mysql> source exercise7.sql;
+------------+
|            |
+------------+
| exercise 7 |
+------------+
1 row in set (0.01 sec)

+-------------+-------------+-----------------+---------------+
| category    | order_count | total_units_sold | total_revenue |
+-------------+-------------+-----------------+---------------+
| Electronics |           4 |              10 |      11900.00 |
| Stationery  |           1 |               5 |        500.00 |
+-------------+-------------+-----------------+---------------+
2 rows in set (0.02 sec)

mysql>
mysql>
```

```sql
-- ### Exercise 8: JOINs with WHERE and HAVING
-- Find customers who have placed more than 1 completed order and their total spending. Display customer ID, customer name,
count of completed orders, and total spent. Order by total spent in descending order.
-- **Hint:** Filter for completed orders using WHERE, then use HAVING to filter groups with more than 1 order.
-- **Expected columns:** customer_id, customer_name, completed_orders, total_spent


select 'exercise 8' as '';
select c.customer_id as customer_id,
c.customer_name as customer_name,
count(o.order_id) as completed_orders,
sum(o.total_amount) as total_spent
from customers c
join orders o on c.customer_id=o.customer_id
where o.status='Completed'
group by c.customer_id,c.customer_name
having count(o.order_id)>1
order by total_spent desc;
```

exercise8.sql × SQL Database.sql

exercise8.sql

```sql
-- ### Exercise 8: JOINs with WHERE and HAVING
-- Find customers who have placed more than 1 completed order and their total spending. Display customer ID, customer name, count of completed orders,
--  and total spent. Order by total spent in descending order.

-- **Hint:** Filter for completed orders using WHERE, then use HAVING to filter groups with more than 1 order.
-- **Expected columns:** customer_id, customer_name, completed_orders, total_spent
select 'exercise 8' as '';
select c.customer_id as customer_id,
c.customer_name as customer_name,
count(o.order_id) as completed_orders,
sum(o.total_amount) as total_spent
from customers c
join orders o on c.customer_id=o.customer_id
where o.status='Completed'
group by c.customer_id,c.customer_name
having count(o.order_id)>1
order by total_spent desc;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL wsl - JC

```
mysql> source exercise8.sql;
+------------+
|            |
+------------+
| exercise 8 |
+------------+
1 row in set (0.00 sec)

+-------------+---------------+-----------------+-------------+
| customer_id | customer_name | completed_orders | total_spent |
+-------------+---------------+-----------------+-------------+
|           1 | Amit Sharma   |               2 |     4300.00 |
|           3 | Rahul Mehta   |               2 |     3600.00 |
+-------------+---------------+-----------------+-------------+
2 rows in set (0.00 sec)

mysql>
mysql>
```

```sql
-- ### Exercise 9: Finding Unmatched Records
-- Find all products that have never been ordered. Display product ID, product name, category, and unit price. Order by product ID.
-- **Hint:** Use LEFT JOIN and check for NULL values in the joined table.
-- **Expected columns:** product_id, product_name, category, unit_price
```

```
select 'exercise 9' as '';
select p.product_id as product_id,
p.product_name as product_name,
p.category as category,
p.unit_price as unit_price
from products p
left join order_details od on p.product_id=od.product_id
where od.product_id is null
order by p.product_id;
```

exercise9.sql × SQL Database.sql exercise8.sql

exercise9.sql

```
-- Find all products that have never been ordered. Display product ID, product name, category, and unit price. Order by product ID.

-- **Hint:** Use LEFT JOIN and check for NULL values in the joined table.

-- **Expected columns:** product_id, product_name, category, unit_price
select 'exercise 9' as '';
select p.product_id as product_id,
p.product_name as product_name,
p.category as category,
p.unit_price as unit_price
from products p
left join order_details od on p.product_id=od.product_id
where od.product_id is null
order by p.product_id;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
mysql> source exercise9.sql;
+------------+
|            |
+------------+
| exercise 9 |
+------------+
1 row in set (0.00 sec)

+------------+--------------+------------+------------+
| product_id | product_name | category   | unit_price |
+------------+--------------+------------+------------+
|        205 | Pen          | Stationery |      20.00 |
+------------+--------------+------------+------------+
1 row in set (0.01 sec)
```

```
-- ### Exercise 10: Advanced Multi-Table Analysis
-- For each country, show the total number of orders, total revenue, and average order value. Only include countries that have orders. Display country, total orders, customer count, total revenue,
and average order value. Order by total revenue in descending order.
-- **Hint:** Join customers and orders, group by country, and use HAVING to exclude countries with no orders.
-- **Expected columns:** country, total_orders, customer_count, total_revenue, avg_order_value
SELECT c.country,
COUNT(o.order_id) AS total_orders,
COUNT(DISTINCT c.customer_id) AS customer_count,
SUM(o.total_amount) AS total_revenue,
AVG(o.total_amount) AS avg_order_value
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.country
HAVING COUNT(o.order_id) > 0
ORDER BY total_revenue DESC;
```

GuriviReddyYasaswini.sql exercise10.sql ×

exercise10.sql

```
-- ### Exercise 10: Advanced Multi-Table Analysis
-- For each country, show the total number of orders, total revenue, and average order value. Only include countries that have orders. Display country, total orde

-- **Hint:** Join customers and orders, group by country, and use HAVING to exclude countries with no orders.
-- **Expected columns:** country, total_orders, customer_count, total_revenue, avg_order_value
SELECT c.country,
COUNT(o.order_id) AS total_orders,
COUNT(DISTINCT c.customer_id) AS customer_count,
SUM(o.total_amount) AS total_revenue,
AVG(o.total_amount) AS avg_order_value
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.country
HAVING COUNT(o.order_id) > 0
ORDER BY total_revenue DESC;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL wsl - JOIN, GROUP BY

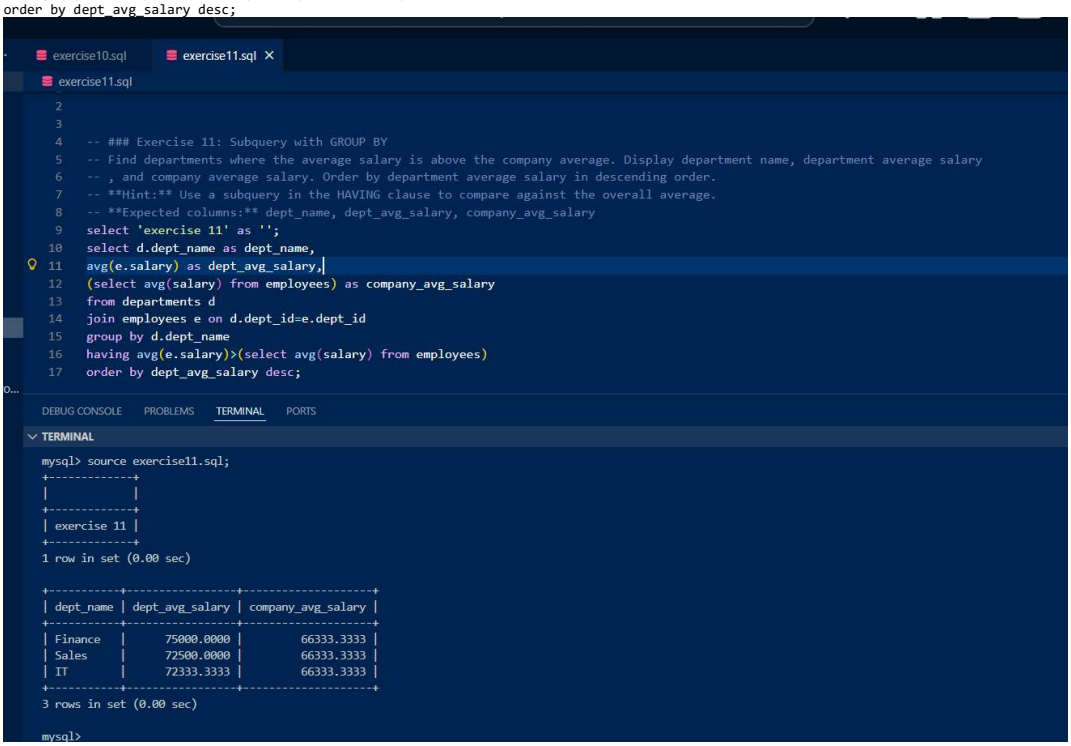
```
mysql>
mysql>
mysql>
mysql>
mysql> source exercise10.sql;
+---------+--------------+----------------+--------------+----------------+
| country | total_orders | customer_count | total_revenue | avg_order_value |
+---------+--------------+----------------+--------------+----------------+
| India   |            5 |              3 |      11100.00 |     2220.000000 |
+---------+--------------+----------------+--------------+----------------+
1 row in set (0.00 sec)

mysql>
mysql>
```

```
-- ### Exercise 11: Subquery with GROUP BY
-- Find departments where the average salary is above the company average. Display department name, department average salary
-- , and company average salary. Order by department average salary in descending order.
```

```
-- **Hint:** Use a subquery in the HAVING clause to compare against the overall average.
-- **Expected columns:** dept_name, dept_avg_salary, company_avg_salary


select 'exercise 11' as '';
select d.dept_name as dept_name,
avg(e.salary) as dept_avg_salary,
(select avg(salary) from employees) as company_avg_salary
from departments d
join employees e on d.dept_id=e.dept_id
group by d.dept_name
having avg(e.salary)>(select avg(salary) from employees)
order by dept_avg_salary desc;
```




```
-- ### Exercise 12: Date-Based Grouping
-- Show monthly order statistics for 2024. Display month (YYYY-MM format), order count, monthly revenue, and average order value. Order by month.
-- **Hint:** Use date functions to extract year-month. Functions vary by database:
-- - SQLite: `strftime('%Y-%m', order_date)`
-- - MySQL: `DATE_FORMAT(order_date, '%Y-%m')`
-- - PostgreSQL: `TO_CHAR(order_date, 'YYYY-MM')`
-- **Expected columns:** month, order_count, monthly_revenue, avg_order_value




select 'exercise 12' as '';
select date_format(order_date,'%Y-%m') as month,
count(order_id) as order_count,
sum(total_amount) as monthly_revenue,
avg(total_amount) as avg_order_value
from orders
where year(order_date)=2024
group by date_format(order_date,'%Y-%m')
order by month;
```

exercise12.sql × SQL Database.sql

exercise12.sql

```sql
-- ### Exercise 12: Date-Based Grouping
-- Show monthly order statistics for 2024. Display month (YYYY-MM format), order count, monthly revenue, and average order value. Order by month.
-- **Hint:** Use date functions to extract year-month. Functions vary by database:
-- - SQLite: `strftime('%Y-%m', order_date)`
-- - MySQL: `DATE_FORMAT(order_date, '%Y-%m')`
-- - PostgreSQL: `TO_CHAR(order_date, 'YYYY-MM')`
-- **Expected columns:** month, order_count, monthly_revenue, avg_order_value
select 'exercise 12' as '';
select date_format(order_date,'%Y-%m') as month,
count(order_id) as order_count,
sum(total_amount) as monthly_revenue,
avg(total_amount) as avg_order_value
from orders
where year(order_date)=2024
group by date_format(order_date,'%Y-%m')
order by month;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
mysql> source exercise12.sql;
+-------------+
|             |
+-------------+
| exercise 12 |
+-------------+
1 row in set (0.00 sec)

+---------+-------------+----------------+----------------+
| month   | order_count | monthly_revenue | avg_order_value |
+---------+-------------+----------------+----------------+
| 2024-01 |           1 |        2500.00 |     2500.000000 |
| 2024-02 |           2 |        3300.00 |     1650.000000 |
| 2024-03 |           2 |        5300.00 |     2650.000000 |
+---------+-------------+----------------+----------------+
3 rows in set (0.00 sec)
```

```sql
-- ### Exercise 13: Complex Filtering
-- Find the top 3 products by revenue. Display product ID, product name, category, total quantity sold,
-- total revenue, and times ordered. Order by total revenue in descending order and limit to 3 results.
-- **Hint:** Calculate revenue as quantity × unit_price, use ORDER BY and LIMIT.
-- **Expected columns:** product_id, product_name, category, total_quantity, total_revenue, times_ordered



select 'exercise 13' as '';
select p.product_id as product_id,
p.product_name as product_name,
p.category as category,
sum(od.quantity) as total_quantity,
sum(od.quantity*od.unit_price) as total_revenue,
count(distinct od.order_id) as times_ordered
from products p
join order_details od on p.product_id=od.product_id
group by p.product_id,p.product_name,p.category
order by total_revenue desc
limit 3;
```

SQL Database.sql exercise13.sql ×

exercise13.sql

```sql

-- ### Exercise 13: Complex Filtering
-- Find the top 3 products by revenue. Display product ID, product name, category, total quantity
-- sold, total revenue, and times ordered. Order by total revenue in descending order and limit to 3 results.

-- **Hint:** Calculate revenue as quantity × unit_price, use ORDER BY and LIMIT.
-- **Expected columns:** product_id, product_name, category, total_quantity, total_revenue, times_ordered
select 'exercise 13' as '';
select p.product_id as product_id,
p.product_name as product_name,
p.category as category,
sum(od.quantity) as total_quantity,
sum(od.quantity*od.unit_price) as total_revenue,
count(distinct od.order_id) as times_ordered
from products p
join order_details od on p.product_id=od.product_id
group by p.product_id,p.product_name,p.category
order by total_revenue desc
limit 3;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
1 row in set (0.00 sec)
| exercise 13 |
+-------------+
1 row in set (0.00 sec)
+-------------+
1 row in set (0.00 sec)
1 row in set (0.00 sec)

+------------+--------------+-------------+----------------+---------------+---------------+
| product_id | product_name | category    | total_quantity | total_revenue | times_ordered |
+------------+--------------+-------------+----------------+---------------+---------------+
|        203 | Monitor      | Electronics |              1 |       8000.00 |             1 |
|        201 | Keyboard     | Electronics |              6 |       3000.00 |             3 |
|        202 | Mouse        | Electronics |              3 |        900.00 |             2 |
+------------+--------------+-------------+----------------+---------------+---------------+
3 rows in set (0.00 sec)
```

```sql
-- ### Exercise 14: CASE with GROUP BY
-- Categorize employees by salary range and count employees in each range by department. Salary ranges:
-- - Entry Level: Less than 60000
```

```
-- - Mid Level: 60000 to 74999
-- - Senior Level: 75000 and above
-- Display department name, salary range, employee count, and average salary. Order by department name and then average salary.
-- **Hint:** Use CASE statement to create salary ranges, include it in both SELECT and GROUP BY.
-- **Expected columns:** dept_name, salary_range, employee_count, avg_salary
```

```
-- ### Exercise 15: Comprehensive Report
-- Create a comprehensive customer report for customers with completed orders. Display: customer ID, customer name, country, total orders, total spent, average order value, last order date, and
first order date. Order by total spent in descending order.
-- **Hint:** Join customers and orders, filter for completed status, use multiple aggregate functions.
-- **Expected columns:** customer_id, customer_name, country, total_orders, total_spent, avg_order_value, last_order_date, first_order_date
```

```
select 'exercise 15' as '';
SELECT c.customer_id,
c.customer_name,
c.country,
COUNT(o.order_id) AS total_orders,
SUM(o.total_amount) AS total_spent,
AVG(o.total_amount) AS avg_order_value,
MAX(o.order_date) AS last_order_date,
MIN(o.order_date) AS first_order_date
FROM customers c
JOIN orders o ON c.customer_id=o.customer_id
WHERE o.status='Completed'
GROUP BY c.customer_id,c.customer_name,c.country
ORDER BY total_spent DESC;
```

SQL Database.sql | exercise15.sql

exercise15.sql

```
-- **Hint:** Join customers and orders, filter for completed status, use multiple aggregate functions.
-- **Expected columns:** customer_id, customer_name, country, total_orders, total_spent, avg_order_value, last_order_date, first_order_date

select 'exercise 15' as '';
SELECT c.customer_id,
c.customer_name,
c.country,
COUNT(o.order_id) AS total_orders,
SUM(o.total_amount) AS total_spent,
AVG(o.total_amount) AS avg_order_value,
MAX(o.order_date) AS last_order_date,
MIN(o.order_date) AS first_order_date
FROM customers c
JOIN orders o ON c.customer_id=o.customer_id
WHERE o.status='Completed'
GROUP BY c.customer_id,c.customer_name,c.country
ORDER BY total_spent DESC;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
mysql>
mysql> source exercise15.sql;
+-------------+
|             |
+-------------+
| exercise 15 |
+-------------+
1 row in set (0.00 sec)

+-------------+---------------+---------+--------------+-------------+-----------------+-----------------+------------------+
| customer_id | customer_name | country | total_orders | total_spent | avg_order_value | last_order_date | first_order_date |
+-------------+---------------+---------+--------------+-------------+-----------------+-----------------+------------------+
|           1 | Amit Sharma   | India   |            2 |     4300.00 |     2150.000000 | 2024-02-15      | 2024-01-10       |
|           3 | Rahul Mehta   | India   |            2 |     3600.00 |     1800.000000 | 2024-03-22      | 2024-02-20       |
+-------------+---------------+---------+--------------+-------------+-----------------+-----------------+------------------+
2 rows in set (0.00 sec)
```

```
-- ## Challenge Exercises (Bonus)
-- These exercises are optional and more advanced. Attempt them if you finish the main exercises early.
-- ### Challenge 1: Employee Hierarchy Analysis
-- Show each employee's level in the organization hierarchy and their path from the top. This requires a recursive query.
-- **Hint:** Use a recursive CTE (Common Table Expression) with WITH RECURSIVE.

-- **Expected columns:** employee_id, employee_name, manager_id, level, path
```

```
-- ### Challenge 2: Product Affinity Analysis
-- Find products that are frequently bought together (appear in the same order at least twice). Display the two product names and how many times they were bought together.
-- **Hint:** Self-join order_details on order_id, ensure product_id1 < product_id2 to avoid duplicates.

-- **Expected columns:** product_1, product_2, times_bought_together
select 'challenge 2' as '';
select p1.product_name as product_1,
p2.product_name as product_2,
count(*) as times_bought_together
from order_details od1
join order_details od2
on od1.order_id=od2.order_id
and od1.product_id<od2.product_id
join products p1 on od1.product_id=p1.product_id
join products p2 on od2.product_id=p2.product_id
group by p1.product_name,p2.product_name
```

```
having count(*)>=2;
```

challenge2.sql

```sql
-- ### Challenge 2: Product Affinity Analysis
-- Find products that are frequently bought together (appear in the same order at least twice). Display the two product names and how

-- **Hint:** Self-join order_details on order_id, ensure product_id1 < product_id2 to avoid duplicates.


-- **Expected columns:** product_1, product_2, times_bought_together

select 'challenge 2' as '';

select p1.product_name as product_1,
p2.product_name as product_2,
count(*) as times_bought_together
from order_details od1
join order_details od2
on od1.order_id=od2.order_id
and od1.product_id<od2.product_id
join products p1 on od1.product_id=p1.product_id
join products p2 on od2.product_id=p2.product_id
group by p1.product_name,p2.product_name
having count(*)>=2;
```

DEBUG CONSOLE PROBLEMS TERMINAL PORTS

TERMINAL

```
mysql> source challenge2.sql;
+-------------+
|             |
+-------------+
| challenge 2 |
+-------------+
1 row in set (0.00 sec)

+----------+----------+----------------------+
| product_1 | product_2 | times_bought_together |
+----------+----------+----------------------+
| Keyboard  | Mouse     |                    2 |
+----------+----------+----------------------+
1 row in set (0.01 sec)
```

```
-- ## Tips for Success
-- 1. **Read the requirements carefully** - Make sure you understand what data is requested
-- 2. **Start with the FROM clause** - Identify which tables you need
-- 3. **Add JOINs one at a time** - Build your query incrementally
-- 4. **Test your query** - Run it to see if you get the expected columns
-- 5. **Check your results** - Do the numbers make sense?
-- 6. **Use table aliases** - Makes queries more readable (e.g., `FROM employees e`)
-- 7. **Format your SQL** - Use proper indentation and line breaks
-- 8. **Comment your code** - Add comments to explain complex logic
-- ## Common Mistakes to Avoid
-- - Forgetting to include columns in GROUP BY that appear in SELECT
-- - Using WHERE instead of HAVING for filtering aggregated results
-- - Incorrect JOIN conditions leading to Cartesian products
-- - Not using LEFT JOIN when you need to include unmatched records
-- - Forgetting to order results when specified
-- ## Submission Guidelines
-- 1. Save your queries in a file named `firstname_lastname_sql_lab.sql`
-- 2. Include comments with your name and the exercise number
-- 3. Format your SQL code properly with indentation
-- 4. Test all queries before submission
-- 5. Submit your file to your trainer
-- ## Evaluation Criteria
-- Your work will be evaluated on:
-- - **Correctness (60%)** - Does the query return the expected results?
-- - **Query Structure (20%)** - Proper use of JOINs, GROUP BY, and HAVING?
-- - **Efficiency (10%)** - Is the query reasonably optimized?
-- - **Code Quality (10%)** - Is the SQL clean, readable, and well-formatted?
-- Good luck with your exercises!
```